TEMA 3
ANDROID STUDIO Y PRIMERA APP CON JETPACK COMPOSE

Programación Multimedia y Dispositivos Móviles
Versión ALUMNADO

En este tema dejamos de utilizar Kotlin como un programa aislado y empezamos a trabajar dentro de un proyecto Android real. Aprenderás a reconocer Android Studio, crear un proyecto Compose, ejecutar la app, modificar MainActivity, crear funciones @Composable, utilizar @Preview y localizar errores.

Índice

3.1 Preparar Android Studio

3.2 Crear un proyecto Compose

3.3 Entender la ventana de Android Studio

3.4 Estructura del proyecto

3.5 AndroidManifest.xml

3.6 MainActivity y ciclo de arranque

3.7 setContent y MaterialTheme

3.8 Primera función @Composable

3.9 @Preview

3.10 Modificadores básicos

3.11 Device Manager y AVD

3.12 Ejecutar con Run

3.13 Dispositivo físico

3.14 Logcat

3.15 Debugger y breakpoints

3.16 Recursos del proyecto

3.17 XML frente a Compose

3.18 Práctica guiada

3.19 Tres ejercicios integradores

3.1 Preparar Android Studio

Android Studio es el IDE oficial para desarrollar Android. Integra editor de código, Gradle, SDK Manager, emulador, herramientas de depuración, Logcat y herramientas específicas para Jetpack Compose.

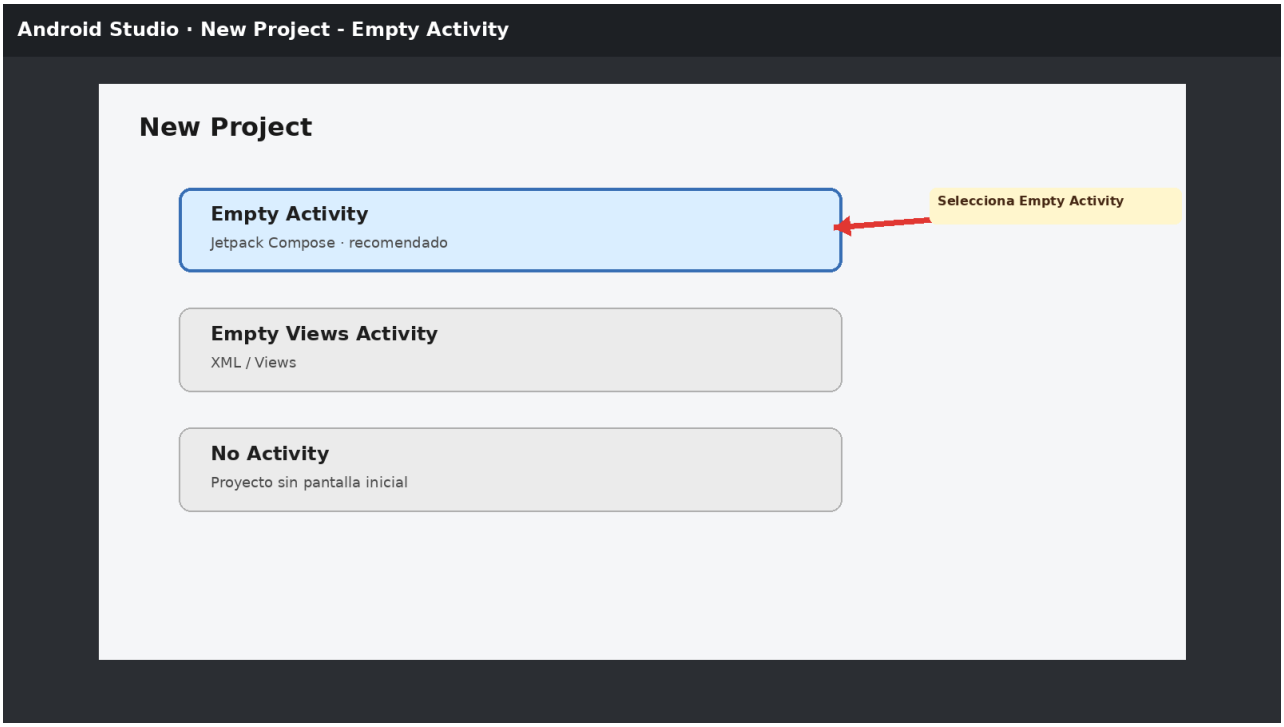
Recomendación para clase. Instala Android Studio en inglés. La documentación, los mensajes de error y la mayoría de tutoriales utilizan los nombres ingleses de menús y herramientas, por lo que resulta más fácil seguirlos.

Durante el Setup Wizard deja instalado Android Emulator y los componentes SDK recomendados. En Windows evita, si puedes, rutas de proyecto o de usuario con caracteres problemáticos; para las prácticas utilizaremos rutas sencillas como C:\Android\Proyectos.

La versión exacta de Android Studio cambia con frecuencia. Los nombres de algunos botones pueden variar ligeramente, pero los conceptos de este tema permanecen.

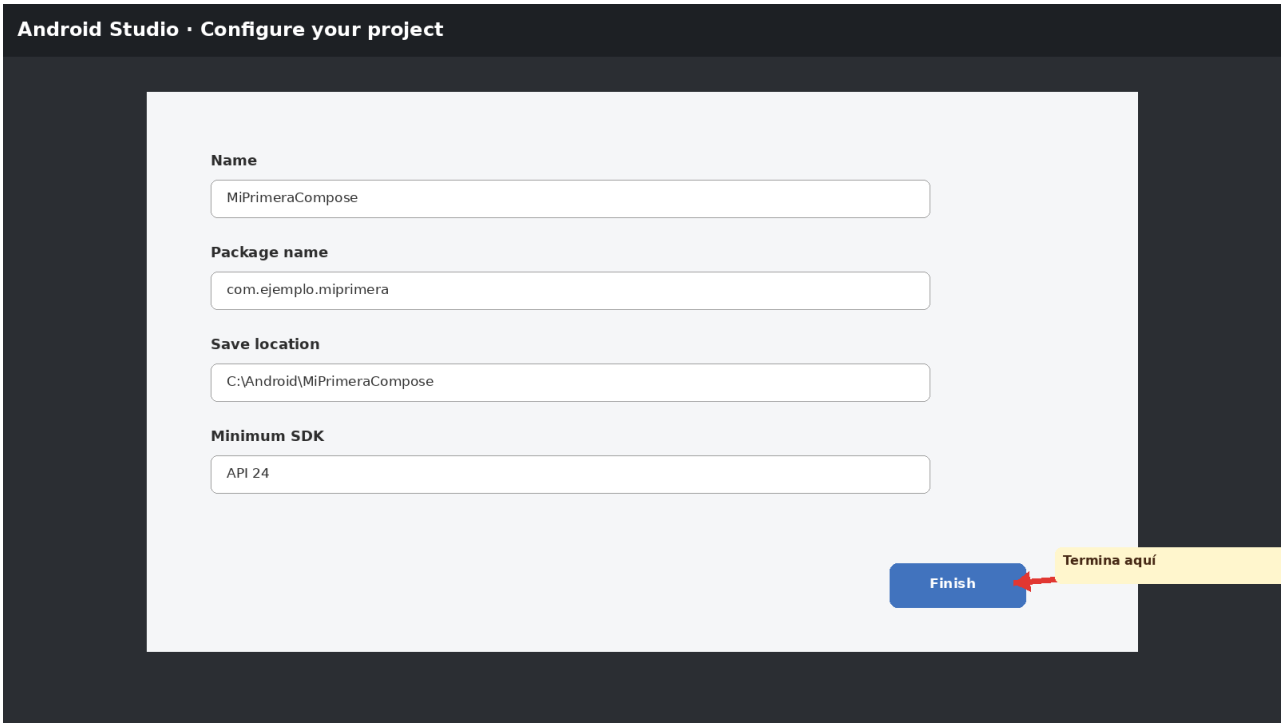
3.2 Crear un proyecto Compose

Desde Welcome to Android Studio selecciona New Project. Para un proyecto nuevo con Jetpack Compose utiliza Empty Activity. No confundas Empty Activity con Empty Views Activity: la segunda corresponde al sistema clásico de Views/XML que vimos en el Tema 0.



Captura didáctica 1. Empty Activity crea la base moderna para trabajar con Jetpack Compose.

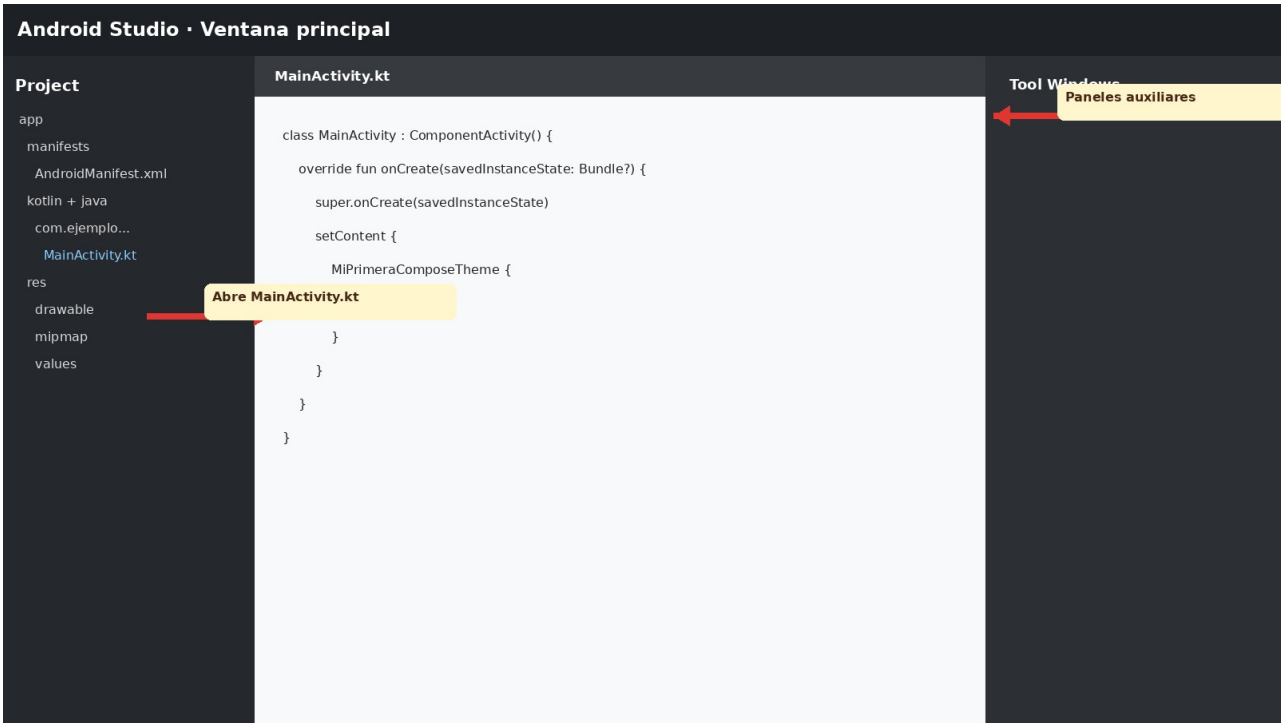
En Configure your project indica Name, Package name, Save location y Minimum SDK. Kotlin es el lenguaje utilizado con Compose. Pulsa Finish y espera a que termine la sincronización de Gradle antes de empezar a modificar archivos.



Captura didáctica 2. Configuración inicial del proyecto. Para clase conviene utilizar nombres y rutas simples.

3.3 Entender la ventana de Android Studio

No intentes aprender todos los botones a la vez. Al principio necesitas dominar cinco zonas: Project para archivos; Editor para código; barra superior para dispositivo y Run; Tool Windows para Logcat, Build o Device Manager; y la zona de Compose Preview cuando trabajemos con @Preview.



Captura didáctica 3. Zonas que debe reconocer el alumno antes de programar.

3.4 Estructura básica del proyecto

Dentro de app encontramos el código y los recursos principales. manifests contiene AndroidManifest.xml; kotlin + java contiene MainActivity.kt y nuestras clases Kotlin; res contiene recursos como imágenes, iconos, textos y temas. Gradle describe cómo se construye el proyecto y qué dependencias utiliza.

Idea importante. No borres archivos porque no sepas para qué sirven. Primero identifica si pertenecen a código Kotlin, recursos, configuración Android o construcción Gradle.

3.5 AndroidManifest.xml

El Manifest declara información esencial de la aplicación: componentes, permisos y características. En un proyecto inicial podrás ver la Activity principal y el intent-filter que permite iniciarla desde el launcher. Más adelante añadiremos permisos cuando una funcionalidad realmente los necesite.

```
<application ...>
  <activity
    android:name=".MainActivity"
    android:exported="true">
    <intent-filter>
      <action android:name="android.intent.action.MAIN" />
      <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
  </activity>
```

```
</application>
```

MAIN + LAUNCHER indica que MainActivity puede ser el punto de entrada que el usuario abre desde el icono de la aplicación.

3.6 MainActivity y el arranque

MainActivity hereda de ComponentActivity. Android llama a onCreate cuando crea la Activity. A diferencia de los programas del Tema 1, no escribimos un main() para iniciar una app Android.

```
class MainActivity : ComponentActivity() {
   override fun onCreate(savedInstanceState: Bundle?) {
      super.onCreate(savedInstanceState)

      setContent {
         PantallaInicio()
      }
   }
}
```

Línea a línea: class MainActivity crea nuestra Activity; : ComponentActivity() indica herencia; override reemplaza el comportamiento de onCreate; super.onCreate(...) permite que la clase padre realice su inicialización; setContent define la interfaz Compose que mostrará esta Activity.

Comparación con Java

```
public class MainActivity extends ComponentActivity {
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      // En proyectos clásicos Java/Views era habitual:
      // setContentView(R.layout.activity_main);
   }
}
```

Compose está pensado para Kotlin. La comparación con Java ayuda a reconocer herencia y override, pero no intentaremos escribir Compose en Java.

3.7 setContent y MaterialTheme

setContent abre el árbol de interfaz de Compose. Dentro solemos aplicar el tema de la aplicación. MaterialTheme proporciona colores, tipografía y formas coherentes a componentes Material como Text o Button.

```
setContent {
    MiPrimeraComposeTheme {
        PantallaInicio()
    }
}
```

Piensa en setContent como el punto en el que decimos: «la pantalla de esta Activity se construye con estas funciones Compose».

3.8 Primera función @Composable

@Composable marca una función que describe una parte de la interfaz. En Compose la UI se construye combinando funciones pequeñas. Una función puede recibir datos y mostrarlos.

```
@Composable
fun Saludo(nombre: String) {
    Text(text = "Hola $nombre")
}
```

Aquí reutilizamos los String Templates del Tema 1. Si llamamos Saludo("Ana"), el componente Text mostrará «Hola Ana». Esta relación entre fundamentos de Kotlin y UI será constante durante el curso.

Comparación conceptual con XML/Java

En Views/XML era habitual declarar un TextView en XML, obtener una referencia y modificar su texto. En Compose describimos directamente el Text desde Kotlin. Esto reduce la separación entre layout y lógica de presentación.

3.9 @Preview: ver la interfaz sin ejecutar toda la app

@Preview indica a Android Studio que renderice una función composable en el IDE. La función de preview también necesita @Composable. Es especialmente útil para cambios visuales rápidos sin arrancar continuamente el emulador.

```
@Preview(showBackground = true)
@Composable
fun SaludoPreview() {
    MiPrimeraComposeTheme {
        Saludo("Ana")
    }
}
```



Captura didáctica 4. @Preview permite comprobar una parte de la UI directamente en Android Studio.

No confundas Preview con Run. Preview ayuda a diseñar, pero debemos ejecutar la aplicación para comprobar el comportamiento real, navegación, permisos y funcionalidades dependientes del dispositivo.

3.10 Modificadores básicos

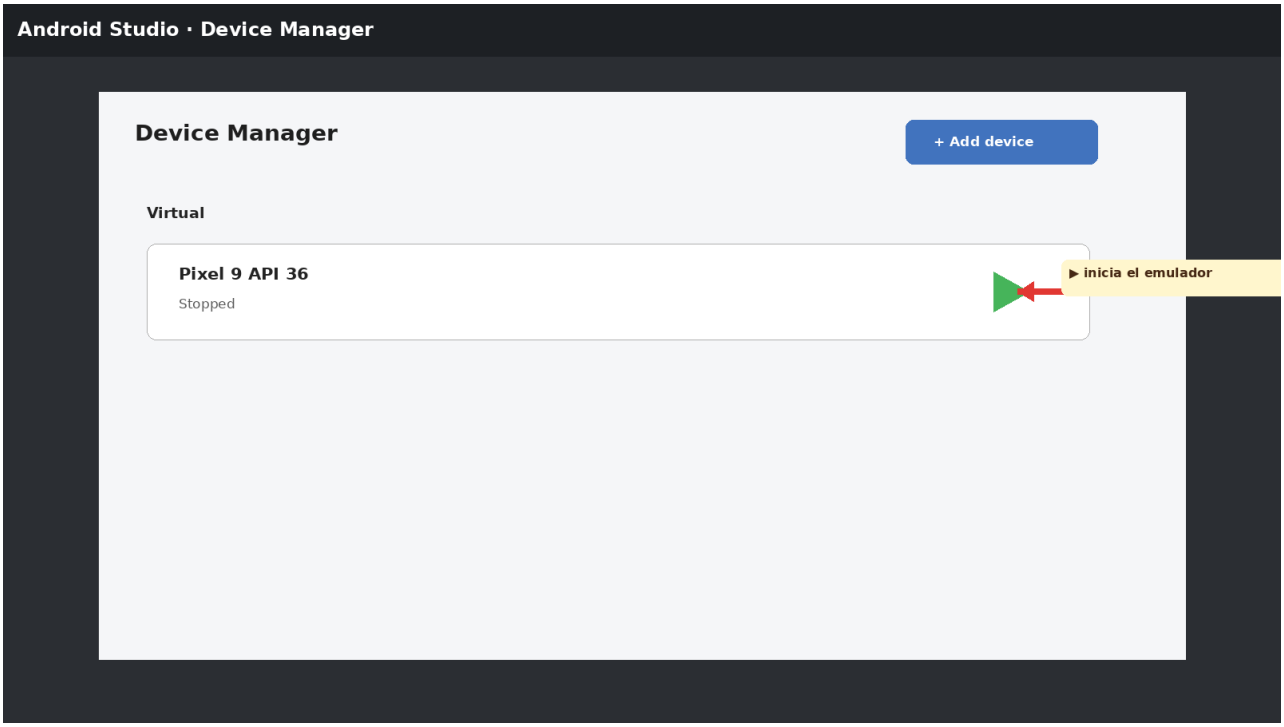
Modifier permite cambiar tamaño, separación, posición, clic y otras características de un composable. El orden de los modificadores puede cambiar el resultado.

```
@Composable
fun TarjetaAlumno(nombre: String) {
  Text(
    text = "Alumno: $nombre",
    modifier = Modifier
      .fillMaxWidth()
      .padding(16.dp)
  )
}
```

fillMaxWidth solicita ocupar el ancho disponible. padding añade espacio alrededor del contenido. dp es una unidad adaptada a densidades de pantalla.

3.11 Device Manager y AVD

Un AVD es un Android Virtual Device: una configuración de teléfono o tablet virtual que Android Emulator ejecuta en el ordenador. Abre Device Manager, crea un dispositivo si no existe, selecciona una imagen de sistema y arráncalo con ▶️.

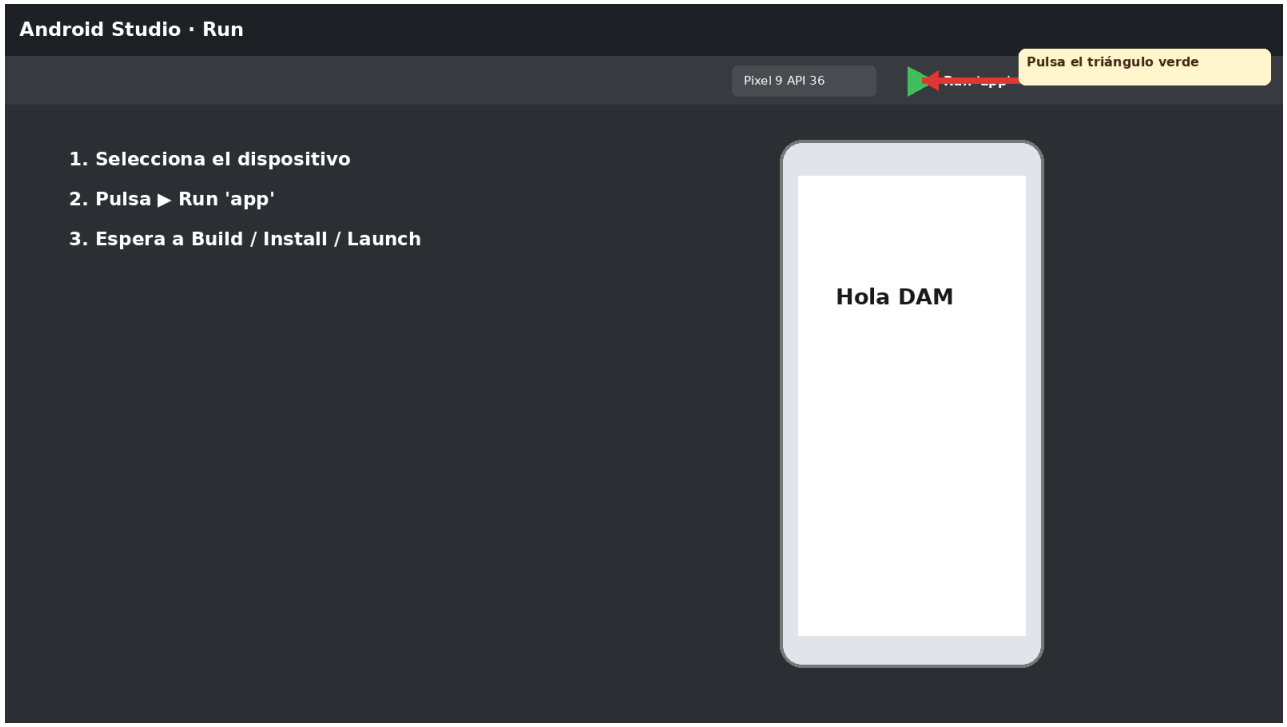


Captura didáctica 5. Device Manager permite crear, iniciar y administrar dispositivos virtuales.

No necesitas crear un emulador nuevo para cada práctica. Mantén uno funcional para clase. Un emulador consume memoria y almacenamiento, por lo que evita ejecutar varios sin necesidad.

3.12 Ejecutar con Run

Selecciona un dispositivo en la barra superior y pulsa ▶️ Run 'app'. Android Studio compila, instala el APK de depuración y abre la app. La primera compilación suele tardar más.



Captura didáctica 6. Flujo: dispositivo → Run → Build → instalación → aplicación ejecutándose.

Si Run está deshabilitado o falla, revisa primero que Gradle Sync haya terminado, que exista un dispositivo disponible y que el panel Build no muestre errores.

3.13 Ejecutar en un dispositivo físico

También podemos probar en un móvil real. Activa las Opciones de desarrollador y la depuración USB, conecta el dispositivo y autoriza el ordenador cuando Android lo solicite. Android 11 o posterior también permite depuración inalámbrica en configuraciones compatibles.

Buena práctica. El emulador facilita probar tamaños y versiones diferentes, pero antes de publicar una app conviene probar también en hardware real.

3.14 Logcat

Logcat muestra mensajes del sistema y de nuestra aplicación. Podemos escribir mensajes propios con Log.d, Log.i, Log.w o Log.e y utilizar filtros para encontrar lo importante.

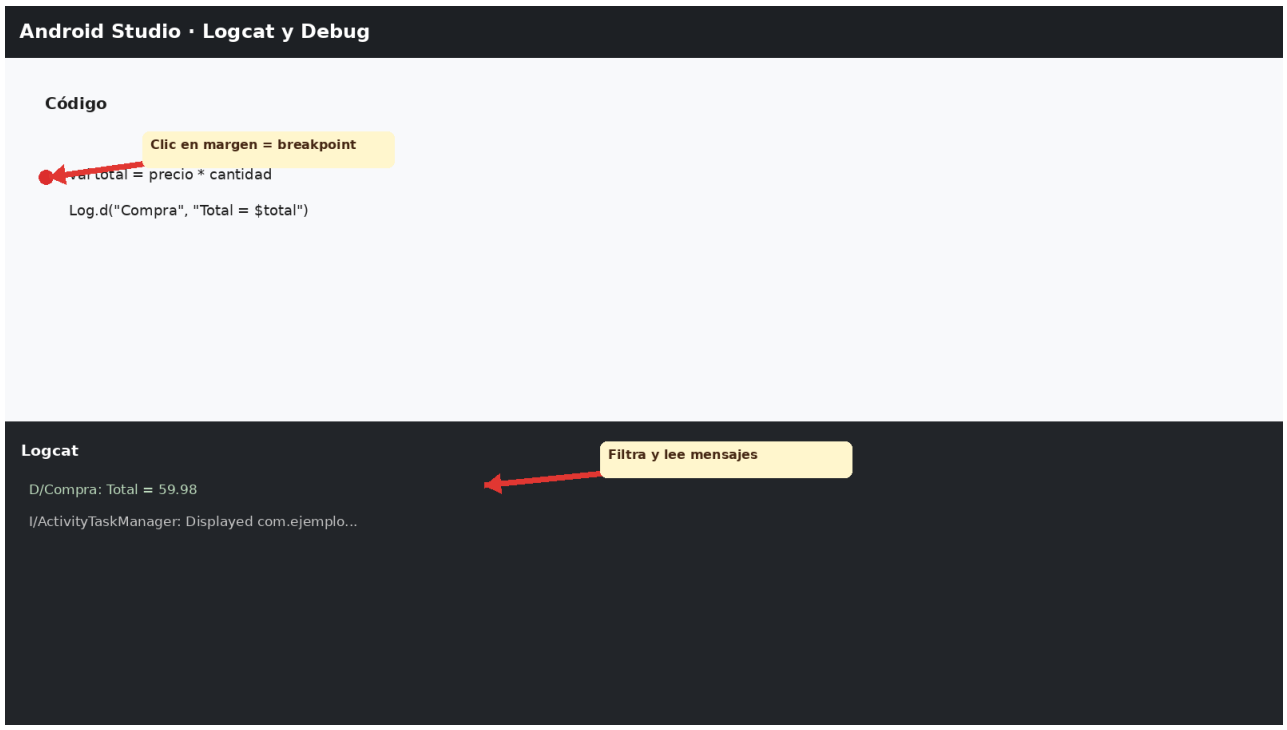
```
val precio = 29.99
val cantidad = 2
val total = precio * cantidad

Log.d("Compra", "Total calculado = $total")
```

La etiqueta «Compra» nos permite localizar el mensaje. No utilices Logcat para imprimir contraseñas, tokens u otros datos sensibles.

3.15 Debugger y breakpoints

Un breakpoint pausa la ejecución en una línea para inspeccionar variables. Haz clic en el margen izquierdo del editor para crear el punto rojo. Ejecuta con Debug en lugar de Run y, cuando se alcance esa línea, examina los valores y avanza paso a paso.



Captura didáctica 7. Breakpoint en el editor y salida de Logcat.

Para aprender programación, el debugger es mucho mejor que adivinar qué ocurre. Si una variable contiene un valor inesperado, puedes localizar exactamente dónde cambia.

3.16 Recursos del proyecto

Aunque Compose construye la interfaz en Kotlin, seguimos utilizando res. Las imágenes pueden almacenarse en drawable, los iconos de launcher en mipmap y los textos reutilizables en values/strings.xml. Evitar textos repetidos directamente en el código facilita traducción y mantenimiento.

```
// res/values/strings.xml
<string name="bienvenida">Bienvenido a DAM Tasks</string>

// Compose
Text(text = stringResource(R.string.bienvenida))
```

3.17 XML frente a Jetpack Compose

XML/Views y Compose son dos formas de construir interfaces Android. XML utiliza archivos de layout y objetos View; Compose utiliza funciones Kotlin declarativas. En el Tema 0 aprendimos XML para entender el sistema clásico y manejar el editor visual. Desde este tema el itinerario principal será Compose.

No significa que XML haya desaparecido: existe mucho software Android basado en Views y pueden convivir ambos sistemas. Para proyectos nuevos, Compose es el enfoque moderno que seguiremos.

3.18 Práctica guiada: nuestra primera pantalla Compose

Objetivo: crear un proyecto nuevo, reconocer sus archivos, sustituir la interfaz inicial por una pantalla propia, verla con Preview y ejecutarla en un emulador.

1. Crear el proyecto. New Project → Empty Activity → Name: PrimeraComposeDAM → Finish. Espera a que Gradle termine.

2. Abrir MainActivity.kt. En Project abre app → kotlin + java → tu paquete → MainActivity.kt.

3. Crear la pantalla. Debajo de MainActivity crea la función PantallaInicio mostrada a continuación.

4. Conectarla a setContent. Dentro del tema de setContent llama a PantallaInicio().

5. Crear Preview. Añade PantallaInicioPreview con @Preview y @Composable.

6. Comprobar Preview. Abre Split/Design y espera al render. Si no aparece, pulsa Build & Refresh en Preview.

7. Iniciar emulador. Tools → Device Manager → inicia tu AVD con ▶.

8. Ejecutar. Selecciona el AVD en la barra superior y pulsa ▶️ Run 'app'.

9. Verificar. La pantalla debe mostrar título, texto y botón. El botón todavía no necesita realizar una acción.

```
@Composable
fun PantallaInicio() {
  Column(
    modifier = Modifier
      .fillMaxSize()
      .padding(24.dp)
  ) {
    Text(
      text = "DAM Tasks",
      style = MaterialTheme.typography.headlineMedium
```

```
        )

        Spacer(modifier = Modifier.height(16.dp))

        Text(text = "Mi primera pantalla con Jetpack Compose")

        Spacer(modifier = Modifier.height(24.dp))

        Button(onClick = { }) {
            Text("Empezar")
        }
    }
}

@Preview(showBackground = true)
@Composable
fun PantallaInicioPreview() {
    MiPrimeraComposeTheme {
        PantallaInicio()
    }
}
```

Qué debes entender: Column coloca elementos verticalmente; Modifier configura espacio; Text muestra texto; Spacer crea separación; Button representa una acción y onClick contiene el código que se ejecutará al pulsarlo. En el Tema 4 profundizaremos en todos estos componentes.

3.19 Ejercicios integradores

Ejercicio 1 - Pantalla de perfil

Crea un proyecto Compose con una pantalla PerfilAlumno que reciba nombre, ciclo y curso. Muestra un título y los tres datos. Añade @Preview con datos de prueba. Ejecuta la app y entrega captura de Preview y del emulador.

Entrega: proyecto ejecutable + captura de Preview + captura en emulador + breve explicación de los composables creados.

Ejercicio 2 - Pantalla de producto

Crea una pantalla Producto que muestre nombre, precio y disponibilidad. Utiliza los fundamentos de Kotlin del Tema 1 para construir un texto diferente según haya stock. Añade un Button «Añadir al carrito» aunque todavía no modifique estado.

Entrega: proyecto ejecutable + captura de Preview + captura en emulador + breve explicación de los composables creados.

Ejercicio 3 - Inicio de DAM Tasks

Empieza la futura app DAM Tasks. La pantalla debe mostrar el nombre de la app, un mensaje de bienvenida, el número inicial de tareas y dos botones: «Ver tareas» y «Nueva tarea». Organiza la UI en funciones composables pequeñas, crea Preview y ejecútala.

Entrega: proyecto ejecutable + captura de Preview + captura en emulador + breve explicación de los composables creados.

Fuentes oficiales

Android Studio installation: developer.android.com/studio/install

Create a project: developer.android.com/studio/projects/create-project

Jetpack Compose quick start: developer.android.com/develop/ui/compose/setup

Compose Preview: developer.android.com/develop/ui/compose/tooling/previews

Run on hardware device: developer.android.com/studio/run/device

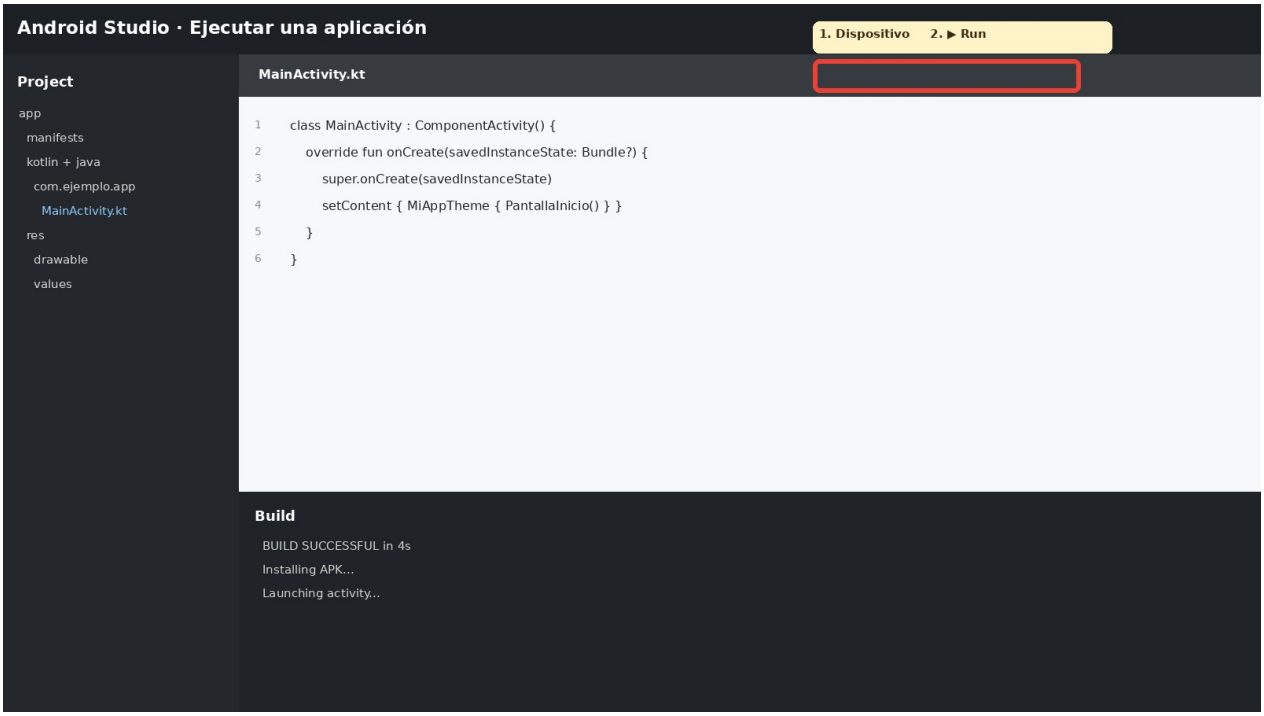
Las capturas de este tema son recreaciones didácticas para destacar los controles que debe localizar el alumnado. La posición exacta de algunos botones puede variar entre versiones de Android Studio.

AMPLIACIÓN DEL TEMA 3 - TRABAJO REAL EN ANDROID STUDIO

Este bloque debe trabajarse antes de pasar al Tema 4. El objetivo es que el alumno sea autónomo para ejecutar, detener, depurar y localizar errores, y que empiece Jetpack Compose desde cero sin limitarse a copiar código.

A. Ejecutar un programa Android: qué ocurre al pulsar Run

Antes de ejecutar, espera a que Android Studio termine la sincronización de Gradle. Después selecciona un emulador o dispositivo físico en la barra superior y pulsa el triángulo verde ▶️ Run 'app'. Android Studio compila el proyecto, genera una versión de depuración, la instala en el dispositivo y abre la Activity principal.



Pantallazo didáctico. Selecciona dispositivo y pulsa Run; el panel Build confirma si la compilación ha terminado correctamente.

Para detener la ejecución utiliza ■ Stop. Si modificas código, vuelve a ejecutar. Si solo has cambiado una parte compatible, Android Studio puede ofrecer Apply Changes, pero al comenzar utilizaremos Run para que el proceso sea más fácil de entender.

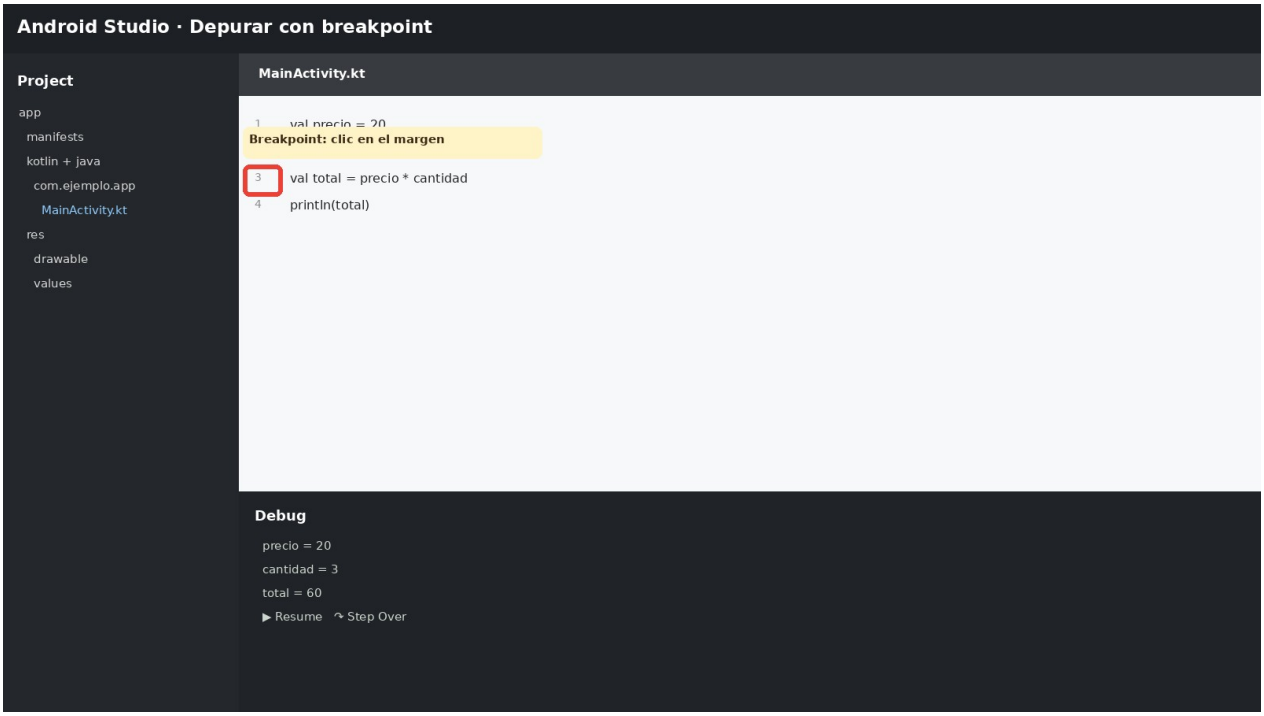
Comprobación del alumno: 1) el panel Build termina sin errores; 2) el emulador muestra la app; 3) puedes cerrar y volver a abrir la aplicación desde su icono.

B. Run no es lo mismo que Debug

Run ejecuta normalmente. Debug ejecuta permitiendo pausar el programa y observar qué está ocurriendo. Un breakpoint es un punto de parada. Se crea haciendo clic en el margen izquierdo, junto al número de línea.

```
val precio = 20
val cantidad = 3
val total = precio * cantidad
println(total)
```

Pon el breakpoint en la línea de total y ejecuta con Debug. Cuando el programa se detenga, observa precio y cantidad. Pulsa Step Over para ejecutar esa línea: ahora total debe valer 60. Resume Program continúa hasta el siguiente breakpoint o hasta finalizar.

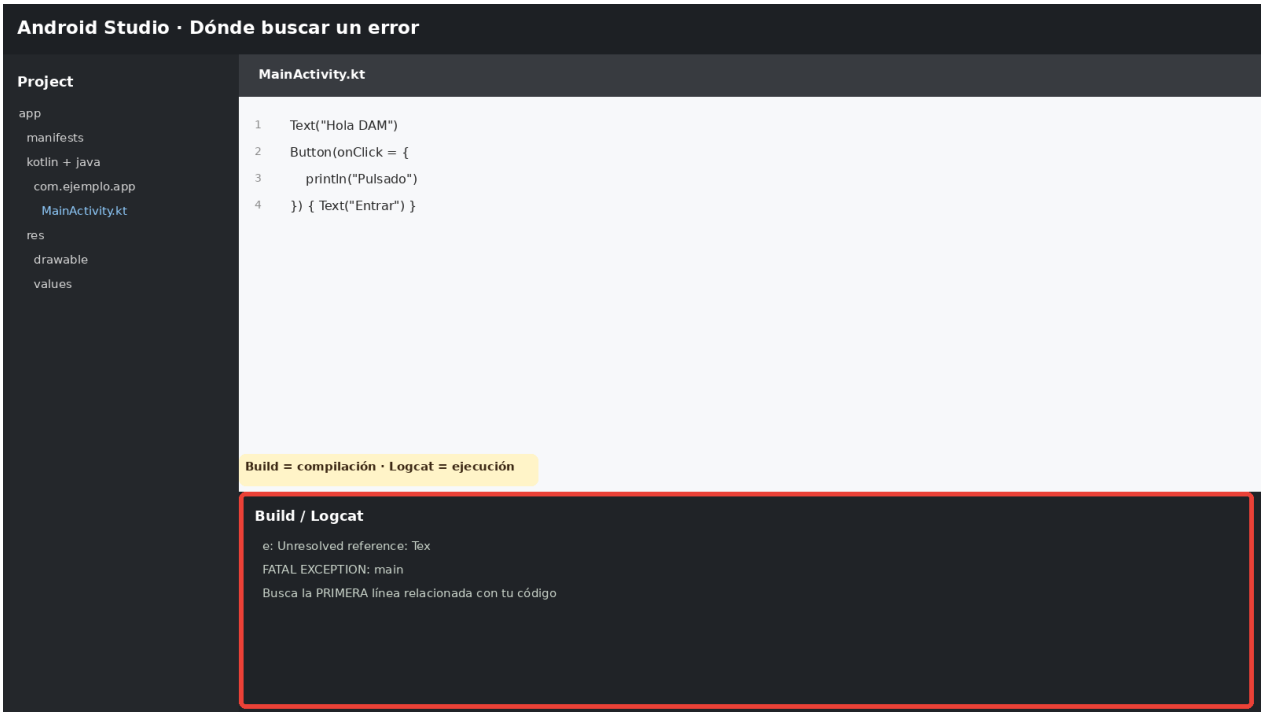


Pantallazo didáctico. Breakpoint, variables y controles básicos del depurador.

Regla de clase: cuando un resultado no sea el esperado, antes de cambiar código al azar, coloca un breakpoint justo antes del cálculo o condición que quieras comprobar.

C. Mi app no funciona: método de diagnóstico

No todos los fallos son iguales. Primero identifica en qué fase aparece el problema. Si el código ni siquiera compila, mira el editor y Build. Si compila pero se cierra o se comporta mal durante la ejecución, mira Logcat y utiliza Debug.



Pantallazo didáctico. Build y Logcat responden a preguntas diferentes.

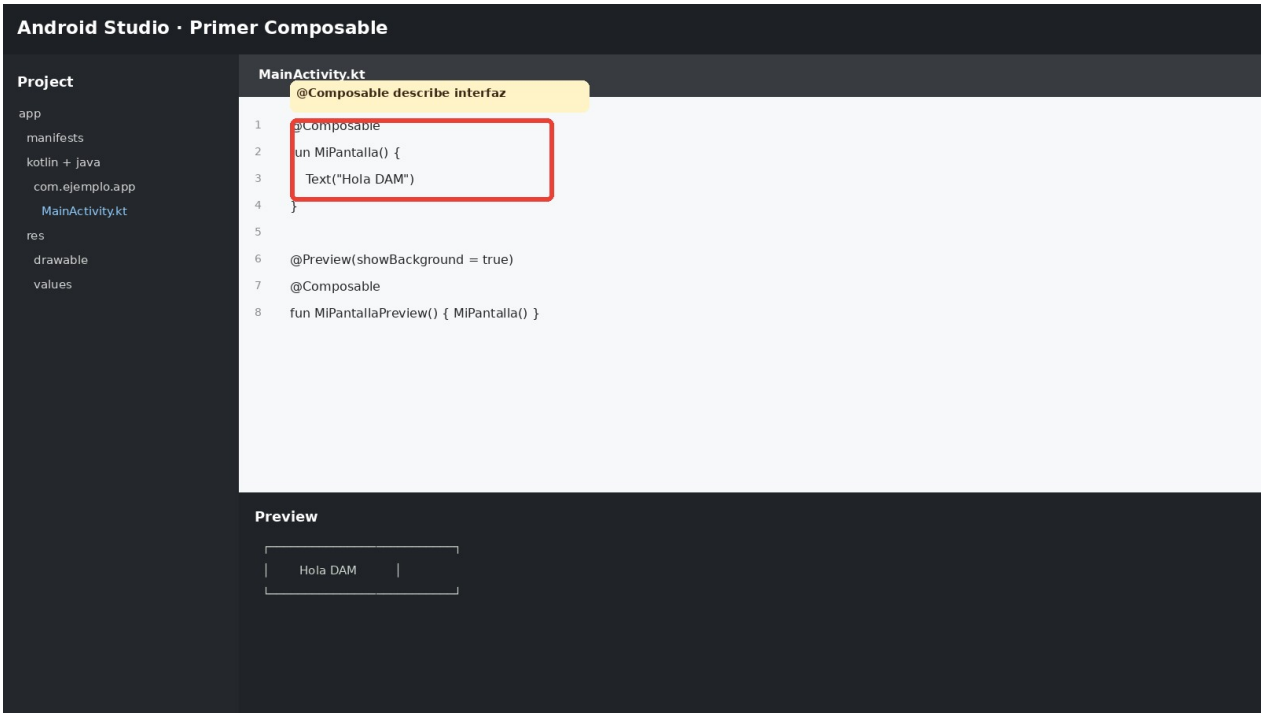
Síntoma	Dónde mirar primero	Acción inicial
Texto rojo / Unresolved reference	Editor + Build	Revisa nombre, import y dependencia.
Gradle Sync failed	Build / notificación de Gradle	Lee el primer error y comprueba conexión/configuración.
No aparece dispositivo	Device Manager	Arranca AVD o reconecta el móvil.
Emulador se queda arrancando	Device Manager	Cold Boot; si persiste, reinicia el AVD.
Preview no aparece	Compose Preview	Build & Refresh; comprueba @Preview y errores.
App se cierra	Logcat	Busca FATAL EXCEPTION y la primera línea de tu paquete.
Composable invocation...	Editor / Build	La función @Composable se está llamando desde un contexto incorrecto.
Paréntesis o llave sin cerrar	Editor	Empareja (), {} y revisa la línea anterior al error.

D. Jetpack Compose desde cero: qué estamos haciendo

Jetpack Compose es el sistema declarativo moderno para construir interfaces Android. En lugar de describir una vista en XML y después buscarla desde el código, escribimos funciones Kotlin que describen qué debe verse para unos datos determinados.

Si creas el proyecto con Empty Activity en una versión actual de Android Studio, el asistente ya prepara Compose. Para empezar en clase NO añadiremos dependencias manualmente: evitamos errores

innecesarios y nos concentramos en comprender la UI. Más adelante veremos dónde se configuran plugins y dependencias.



Pantallazo didáctico. Una función @Composable y su resultado en Preview.

E. Primer paso: mostrar un texto

```
package com.ejemplo.practicacompose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable

class MainActivity : ComponentActivity() {

   override fun onCreate(savedInstanceState: Bundle?) {
      super.onCreate(savedInstanceState)

      // setContent inicia la interfaz creada con Jetpack Compose.
      setContent {
         // Llamamos a la función que dibuja nuestra pantalla.
         MiPantalla()
      }
   }
}

/*
 * @Composable indica que esta función puede crear
 * elementos de la interfaz de usuario (UI).
 */
@Composable
fun MiPantalla() {
```

```
    // Text muestra un texto en la pantalla.
    Text(
        text = "Hola DAM"
    )
}
```

Resultado: aparece el texto «Hola DAM». @Composable indica que la función participa en la construcción de la interfaz; MiPantalla es el nombre que elegimos; Text es un composable ya preparado que muestra texto.

Para verla desde MainActivity:

```
setContent {
    MiAppTheme {
        MiPantalla()
    }
}
```

F. Segundo paso: varios elementos con Column

```
package com.ejemplo.practicacompose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Mostramos nuestra pantalla Compose.
        setContent {
            MiPantalla()
        }
    }
}

@Composable
fun MiPantalla() {

    /*
     * Column coloca los elementos que contiene
     * uno debajo de otro, en vertical.
     */
    Column {

        // Primer texto de la pantalla.
        Text(text = "Programación Multimedia")
```

```
        // Segundo texto.
        Text(text = "2º DAM")

        // Tercer texto.
        Text(text = "Jetpack Compose")
    }
}
```

Column coloca sus hijos uno debajo de otro. Resultado esperado: tres líneas verticales. Si utilizáramos Row, se intentarían colocar horizontalmente.

G. Tercer paso: añadir un botón

```
package com.ejemplo.practicacompose

import android.os.Bundle
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            MiPantalla()
        }
    }
}

@Composable
fun MiPantalla() {

    // Column organiza verticalmente el texto y el botón.
    Column {

        Text(text = "Mi primera aplicación")

        /*
         * Button crea un botón.
         * El bloque onClick se ejecuta cuando lo pulsamos.
         */
        Button(
            onClick = {
                // Escribimos un mensaje que podremos localizar en Logcat.
                Log.d("MiApp", "El alumno ha pulsado Entrar")
            }
        ) {
            // Texto visible dentro del botón.
            Text(text = "Entrar")
        }
```

```
    }
}
```

Button recibe una lambda en onClick: ese bloque se ejecuta cuando el usuario pulsa. El Text interior define lo que se ve dentro del botón. Al pulsarlo, el mensaje se puede localizar en la salida correspondiente; más adelante sustituiremos println por cambios de estado y navegación.

H. Cuarto paso: Modifier y separación

```
package com.ejemplo.practicacompose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            MiPantalla()
        }
    }
}

@Composable
fun MiPantalla() {

    Column(
        modifier = Modifier
            // Ocupa todo el espacio disponible de la pantalla.
            .fillMaxSize()
            // Deja 24 dp de separación respecto a los bordes.
            .padding(24.dp)
    ) {

        Text(text = "DAM Tasks")

        // Spacer crea un espacio vertical de 16 dp.
        Spacer(
            modifier = Modifier.height(16.dp)
        )

        Text(text = "Tienes 3 tareas pendientes")
    }
}
```

Modifier encadena modificaciones. fillMaxSize solicita todo el espacio disponible; padding separa el contenido de los bordes; Spacer crea un hueco entre componentes. El orden de los modificadores puede alterar el resultado.

I. Quinto paso: Preview antes de ejecutar

```
@Preview(showBackground = true)
@Composable
fun MiPantallaPreview() {
  MiAppTheme {
    MiPantalla()
  }
}
```

Preview acelera el diseño, pero no sustituye al emulador. Utiliza Preview para comprobar aspecto y Run para comprobar la aplicación real.

J. Ejemplo completo y resultado esperado

```
@Composable
fun InicioDamTasks(nombre: String, tareas: Int) {
  Column(
    modifier = Modifier
      .fillMaxSize()
      .padding(24.dp)
  ) {
    Text(
      text = "DAM Tasks",
      style = MaterialTheme.typography.headlineLarge
    )
    Spacer(Modifier.height(16.dp))
    Text("Hola, $nombre")
    Text("Tienes $tareas tareas pendientes")
    Spacer(Modifier.height(24.dp))
    Button(onClick = { }) {
      Text("Ver tareas")
    }
    Button(onClick = { }) {
      Text("Nueva tarea")
    }
  }
}
```

Resultado esperado: título DAM Tasks, saludo personalizado, número de tareas y dos botones. Aquí ya estamos reutilizando variables, parámetros y String Templates del Tema 1 dentro de una interfaz Android.

K. Mini práctica de autonomía

Crea un proyecto Empty Activity llamado PracticaCompose01. Haz una pantalla con tu nombre, el ciclo DAM, un contador inicial y un botón. Añade Preview, ejecútala en el emulador, coloca un breakpoint en una línea donde calcules un valor y localiza un mensaje en Logcat. Entrega cuatro capturas: código, Preview, emulador y depurador.

No se evalúa que la pantalla sea bonita todavía. Se evalúa que sepas crearla, ejecutarla, observarla y localizar problemas básicos.

Checklist antes de pasar al Tema 4

☐ Sé crear un proyecto Empty Activity con Compose.

☐ Sé esperar a Gradle Sync y distinguir un error de compilación.

☐ Sé seleccionar un emulador y ejecutar con Run.

☐ Sé detener la app y volver a ejecutarla.

☐ Sé usar un breakpoint y Step Over.

☐ Sé buscar un cierre inesperado en Logcat.

☐ Entiendo qué significa @Composable.

☐ Sé utilizar Text, Column, Button, Spacer y Modifier de forma básica.

☐ Sé crear una función @Preview.

☐ Puedo explicar la diferencia básica entre XML/Views y Compose.

ANEXO - NORMA DE CÓDIGO DEL CURSO

A partir de este tema, los ejemplos principales se presentan con el contexto necesario para poder copiarlos en Android Studio y ejecutarlos. Además, las llamadas nuevas aparecen comentadas con // o /* ... */. No se pretende memorizar los imports: se incluyen para que el ejemplo sea reproducible y Android Studio pueda resolver cada componente.

Ejemplo completo: pantalla, botón, Logcat y Preview

```
package com.ejemplo.tema3

import android.os.Bundle
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        /*
         * setContent indica que la interfaz de esta Activity
         * se va a construir utilizando Jetpack Compose.
         */
        setContent {

            // Llamamos a nuestra función de interfaz.
            PantallaInicio(
                nombre = "Alumno DAM"
            )
        }
    }
}

/*
 * Esta función recibe un nombre y construye
 * la interfaz que verá el usuario.
 */
@Composable
fun PantallaInicio(nombre: String) {

    // Column coloca sus elementos verticalmente.
    Column(
        modifier = Modifier.padding(24.dp)
    ) {
```

```
        // Mostramos un saludo utilizando el parámetro nombre.
        Text(
            text = "Hola, $nombre"
        )

        /*
         * onClick contiene el código que se ejecutará
         * cuando el usuario pulse el botón.
         */
        Button(
            onClick = {
                // Log.d escribe un mensaje de depuración en Logcat.
                Log.d("Tema3", "Se ha pulsado el botón Entrar")
            }
        ) {

            // Texto visible dentro del botón.
            Text(
                text = "Entrar"
            )
        }
    }
}

/*
 * @Preview permite ver esta función en Android Studio
 * sin tener que arrancar el emulador.
 */
@Preview(showBackground = true)
@Composable
fun PantallaInicioPreview() {

    // Usamos datos de prueba únicamente para la Preview.
    PantallaInicio(
        nombre = "Ana"
    )
}
```

Prueba propuesta: ejecuta el proyecto con Run, abre Logcat, pulsa Entrar y busca la etiqueta Tema3. Después coloca un breakpoint dentro de onClick y vuelve a ejecutar con Debug. Así comprobamos en un solo ejemplo UI, ejecución, Logcat y depuración.